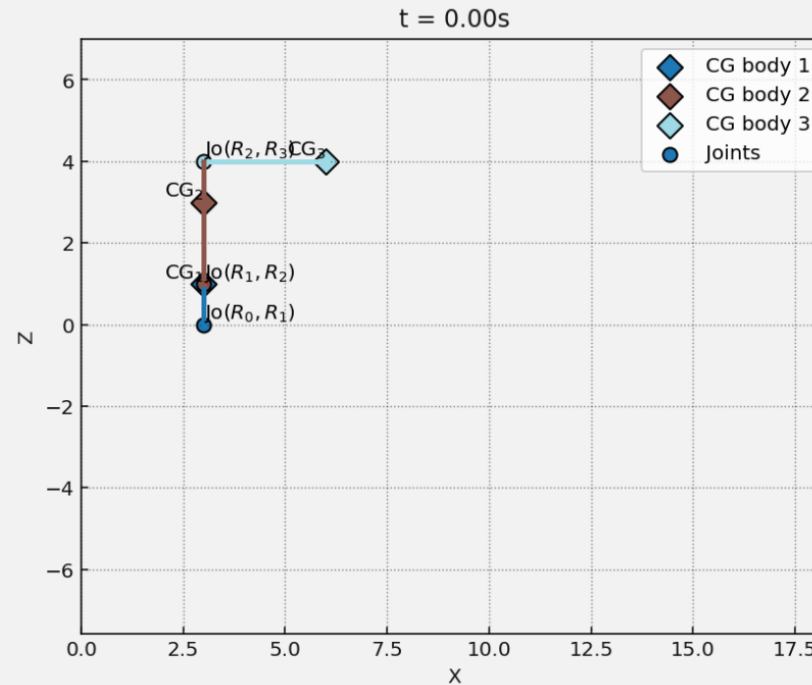


Double pendulum on cart



This example focuses on how to define the bodies in the system and the simulation

Example initialization: do not modify

Do not modify:

- ❖ Defines time as symbolic variable and is used in the checks sections
- ❖ Define the dictionary for points and forces

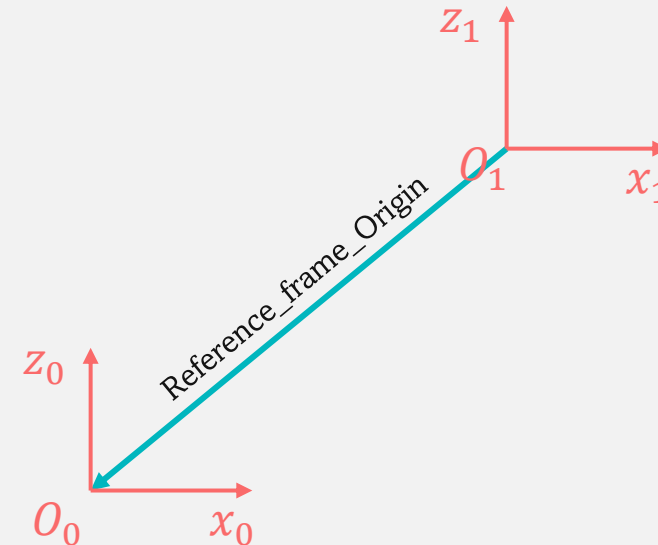
```
# Do not modify  
t = sym.symbols('t', real=True)  
Initial_Points = {}  
Force = {}  
# End of Do not Modify
```

Example initialization: user-defined

Gravity and origin:

- ❖ g is the gravity which can be set to any value, zero is applicable if floating in water
- ❖ `Reference_frame_Origin`: vector from absolute origin to origin of body 1
(important when first body is floating)

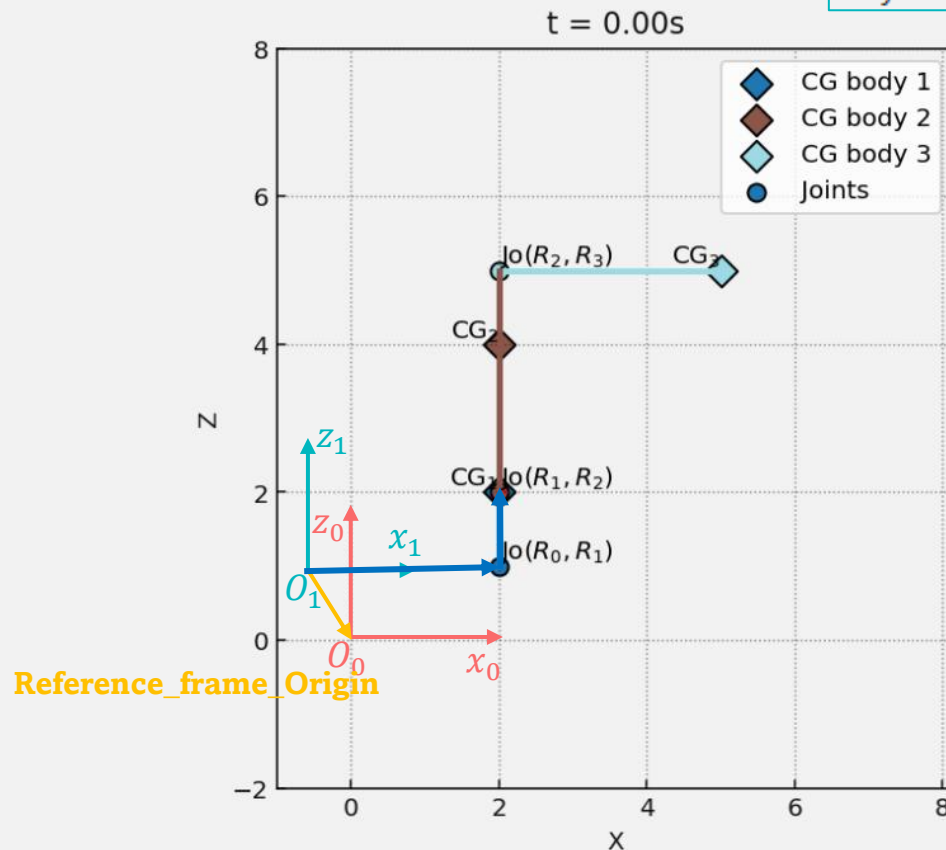
```
# Example 4
g = 9.81 # gravity
Reference_frame_Origin = np.array([1,-1])
Reference_frame_Origin = -Reference_frame_Origin
```



Bodies definition: body 1

- Skip for now

```
# Bodies definition
dataNames = [] # Define symbolic variables for force points (for example, b1, b2, b3).
BodyDataSym = symVars_definition(dataNames, globals()) # DO NOT MODIFY
```



Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

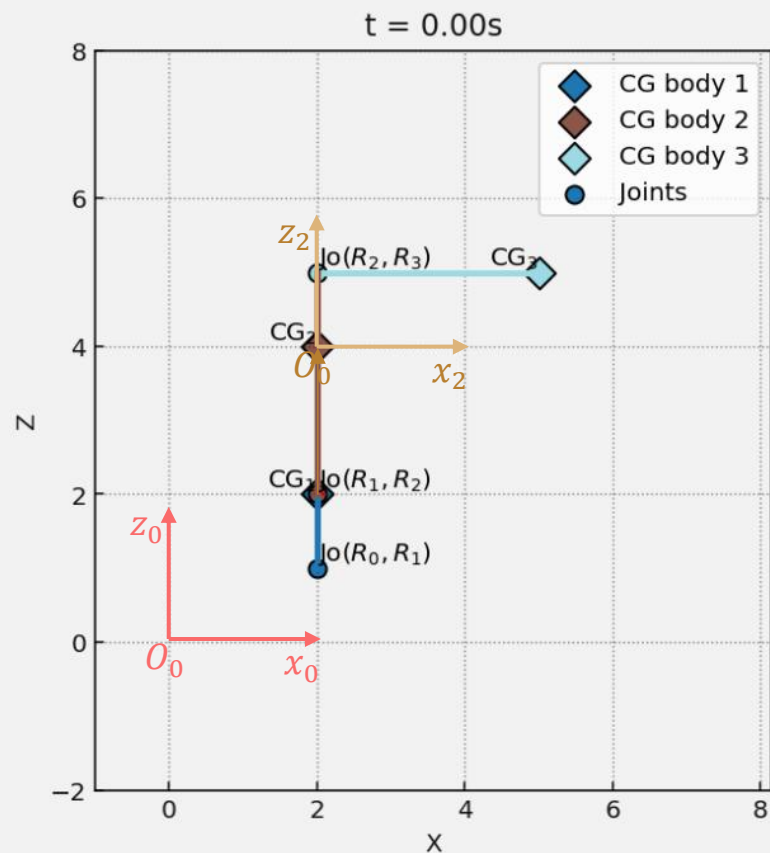
Direction of allowed translation

$S_0^{R_0, R_1}$

$S_1^{R_0, R_1}$

Go to next slide to continue animation

Bodies definition: body 2



Connecting bodies

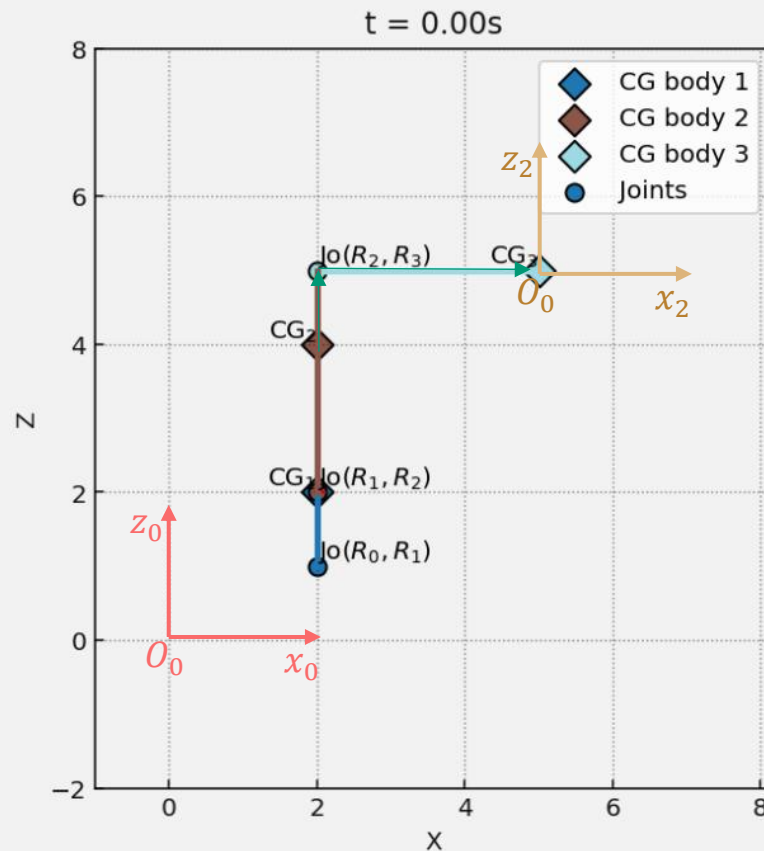
Joint type

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

$S_1^{R_1, R_2}$
 $S_2^{R_1, R_2}$

Go to next slide to continue animation

Bodies definition: body 3



Connecting bodies

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

Joint type

$S_2^{R_2, R_3}$
 $S_3^{R_2, R_3}$

Go to next slide to continue animation

Bodies definition: table display

Summary of how bodies were defined but organized by rows instead of columns

Bodies and joints table:

Joint	Connected Bodies	Joint Type	ParentCGtoJoint	JointtoChildCG	Prismatic	Direction
1	0 1	P	3.000	0.000	0.000	1.000
2	1 2	R	0.000	0.000	0.000	2.000
3	2 3	R	0.000	1.000	3.000	0.000

1.000 0.000

Normalized direction

Simulation and Animation

```
# Simulation
TimeStep    = 0.05
tspan       = 0.05
gVec        = np.ones((len(types),1))
m0          = np.ones(len(types))
J0          = 2.*np.ones(len(types))
```

Factor to apply a different gravity to each body

Mass and Inertia of each body

```
# Animation
animation_on = 0
SaveMovieOn  = None
plotTstep    = 5
```

Boolean to display animation or not

Name with .gif extension if you want to save animation
(requires *animation_on* = 1), **None** otherwise

Number of timeSteps to skip (integer) when
displaying animation (Plots faster)